

Resolução dos Exercícios

Capítulo: O meu primeiro programa

1. Qual função deve estar presente em todos os programas em C?

A função `main`.

2. Como devem terminar todas as instruções em C?

As instruções devem ser finalizadas com ponto e vírgula (`;`).

3. Como é delimitado um bloco em C?

Um bloco é delimitado por um par de chaves (`{ }`).

4. A função `printf` é parte integrante da linguagem C?

Não de maneira estrita. A linguagem C não possui mecanismos incorporados (palavras-chave) para lidar com entrada e saída de dados. Isso fica sob a responsabilidade da Biblioteca Padrão. Embora a função `printf` seja padronizada em todo compilador C, ela precisa ser declarada no código para ser utilizada.

5. Para que serve a linha `#include <stdio.h>` num programa?

A diretiva `#include <stdio.h>` indica ao pré-processador que o arquivo `stdio.h` deve ser incluído no processo de compilação. Esse arquivo contém informações (macros e declarações) voltadas ao tratamento de entrada e saída de dados.

6. A extensão .h indica que o arquivo correspondente é composto por...?

A extensão `.h` indica que o arquivo é um cabeçalho (*header file*), contendo declarações de funções, tipos e macros.

7. Os arquivos com extensão .h são também conhecidos por...?

Header files.

8. Por que razão não se utilizou a linha `#include <stdio.h>` no programa0101.c?

```
main()  
{  
}
```

O programa inicia e finaliza sem realizar qualquer interação de entrada ou saída. Como não utiliza funções como `printf`, a inclusão do cabeçalho é dispensável.

9. Dentro de uma string pode-se usar letras maiúsculas? Justifique.

Sim. Pode-se escrever livremente em uma string. Diferentemente das instruções da linguagem, que devem respeitar regras rígidas por C ser case-sensitive (sensível a maiúsculas e minúsculas), o conteúdo de uma string é tratado apenas como dado e, portanto, não está sujeito a essas restrições.

10. Qual o significado de stdio?.

Standard Input/Output (Entrada e Saída Padrão).

11. Identifique os erros de compilação que seriam detectados nos

seguintes programas:

11.1:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
Main()
{
    printf("Hello World");
}
```

Um programa em C deve, obrigatoriamente, começar pela função `main`. Como a linguagem C é case-sensitive (faz distinção entre maiúsculas e minúsculas), o identificador `Main` escrito no código é considerado diferente da função `main` exigida pelo padrão. Consequentemente, o compilador (ou o linker) não encontrará o ponto de entrada e indicará que o programa não possui uma referência definida para `main`.

11.2:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main
{
    printf("Hello World");
}
```

Entre o nome da função e a abertura do bloco de código, é obrigatório o uso de um par de parênteses `()`. Eles são destinados à indicação de parâmetros (argumentos) da função; se não houver nenhum, permanecem vazios ou com

a palavra `void`. A ausência desses parênteses causará um erro de sintaxe, impedindo a compilação do programa.

11.3:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main()
{
    print ("Hello World");
}
```

A função `print` utilizada no código não está declarada no cabeçalho `stdio.h`; a função correta para a exibição de strings em C é `printf`. Por não estar definida em nenhum local do programa ou em suas bibliotecas incluídas, o uso de `print` causará um erro de referência não encontrada impedindo a conclusão da compilação.

11.4:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main()
{
    printf("Hello") (" World");
}
```

A instrução `printf` deve ser finalizada com um ponto e vírgula (`;`) logo após o fechamento do primeiro par de parênteses. Em vez disso, o compilador encontrará a abertura de um segundo par de parênteses. Como a sintaxe

esperada era um ponto e vírgula para encerrar a instrução, a compilação será interrompida por erro de sintaxe.

11.5:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main()
{
    printf("Hello World");
}
```

O comentário feito no início do código não foi fechado. Logo, todo o restante do código também foi comentado, o que gera um erro de compilação.

11.6:

```
/*
/* Copyright: Asneira Suprema Software!!! */
*/

#include
main()
{
    printf("Hello World");
}
```

Como não existe comentário dentro de comentário, o `*/` logo após o fechamento do primeiro bloco fica 'solto' no código, causando um erro que impede a compilação.

11.7:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main()
{
    printf>Hello World);
}
```

A função `printf` espera receber uma string, cujo conteúdo deve estar envolto em aspas duplas. O código infringe essa regra, causando um erro que torna impossível a compilação do programa.

11.8:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main()
{
    printf>Hello World")
}
```

Como as instruções em C devem ser encerradas com `;`, a omissão desse caractere ao fim do `printf` impedirá a compilação do programa.

11.9:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */
```

```
include
main()
{
    printf("Hello World");
}
```

A diretiva `include` deve ser precedida pelo caractere `#` (`#include`) para sinalizar ao pré-processador a necessidade de incluir o arquivo de cabeçalho especificado.

11.10:

```
/*
 * Copyright: Asneira Suprema Software!!!
 */

#include
main()
{
    printf('Hello World');
}
```

O argumento recebido pela função `printf` está envolto em aspas simples, infringindo a regra de que strings devem utilizar aspas duplas. Isso não necessariamente impedirá a compilação do programa, mas ele terá um comportamento diferente do esperado pelo programador.

12. Os comentários devem ser escritos:

- ☐ a) Antes de qualquer instrução...
- ☐ b) Depois de todas as instruções.
- ☐ c) Antes do main
- ☒ d) **Sempre que o programador ache necessário ou conveniente.**

13. Um programa em C, que tenha comentários no seu código, é, em relação a outro que não os tenha,

- ☐ a) Mais rápido para executar.
- ☐ b) Mais lento para executar.
- ☐ c) Executado praticamente à mesma velocidade, pois os comentários exigem uma utilização ínfima da CPU.
- ☒ **d) Executado à mesma velocidade, pois os comentários são simplesmente ignorados pelo compilador, não havendo qualquer reflexo deles no tempo de execução.**

**14. Indique se são verdadeiras ou falsas as seguintes afirmações:
Os comentários**

- a) só podem executar uma única linha. - **falsa**
- b) podem ocupar várias linhas. - **verdadeira**
- c) podem conter outros comentários dentro. - **falsa**
- d) começam por `/*` e terminam com `*/`. - **verdadeira**
- e) não têm qualquer influência na velocidade de execução de um programa. - **verdadeira**
- f) têm que começar no início de uma linha. - **falsa**
- g) quando ocupam apenas uma linha não precisam terminar com **falsa** (Para o estilo `/* */`, o fechamento é obrigatório mesmo em uma única linha).

15. Escreva um programa que coloque na tela a seguinte frase:

Bem-vindos ao /Mundo\ da programação em "C"

[Ver código-fonte \(exe15.c\)](#)

16. Escreva um programa que coloque na tela uma árvore com o seguinte formato:

```
*
***
```



```
*****
```

```
/|\
```

[Ver código-fonte \(exe16.c\)](#)

17. Escreva um programa que coloque na tela a seguinte saída:

```
Total      =    100%
IVA         =    17%
IRS         =    15%
-----
Líqu.      =    68%
```

[Ver código-fonte \(exe17.c\)](#)

18. Experimente a função `puts("Hello World");` (put string) para escrever a string "Hello World" e indique qual a diferença entre esta e a função `printf`. (Nota: Essa função também faz parte do `stdio.h`):

A função `puts`, diferentemente da `printf`, não permite a formatação do texto, apenas a sua exibição direta; nesse sentido, a função apresenta-se como uma alternativa mais simples. Outro ponto importante é que a `puts` posiciona o cursor em uma nova linha automaticamente após exibir a string, enquanto a `printf` só fará o mesmo se o caractere de nova linha (`\n`) for explicitamente adicionado.

[Ver código-fonte \(exe18.c\)](#)